Član tima: Halilović Elvedina

Odabrana metoda:

```
public List<Knjiga> PretragaKnjiga(string mod, string pojam)
{
    var rezultati = new List<Knjiga>();
    if (IsNullOrWhite(pojam)) { return rezultati; }

    if (mod == "1")
    {
        for (int i = 0; i < _db.knjige.Count; i++)
        {
            var k = _db.knjige[i];
            if (CaseInsensitiveContains(k.NazivKnjige ?? "", pojam))
            {
                rezultati.Add(k);
            }
        }
    }
    else // Autor
    {
        for (int i = 0; i < _db.knjige.Count; i++)
        {
            var k = _db.knjige[i];
            if (k.Autori == null) continue;

            bool match = false;
            for (int a = 0; a < k.Autori.Count; a++)
            {
                var au = k.Autori[a];
                string punoIme = $"{au.ImeAutora} {au.PrezimeAutora}";
                if (CaseInsensitiveContains(punoIme, pojam))
                { match = true; break; }
            }

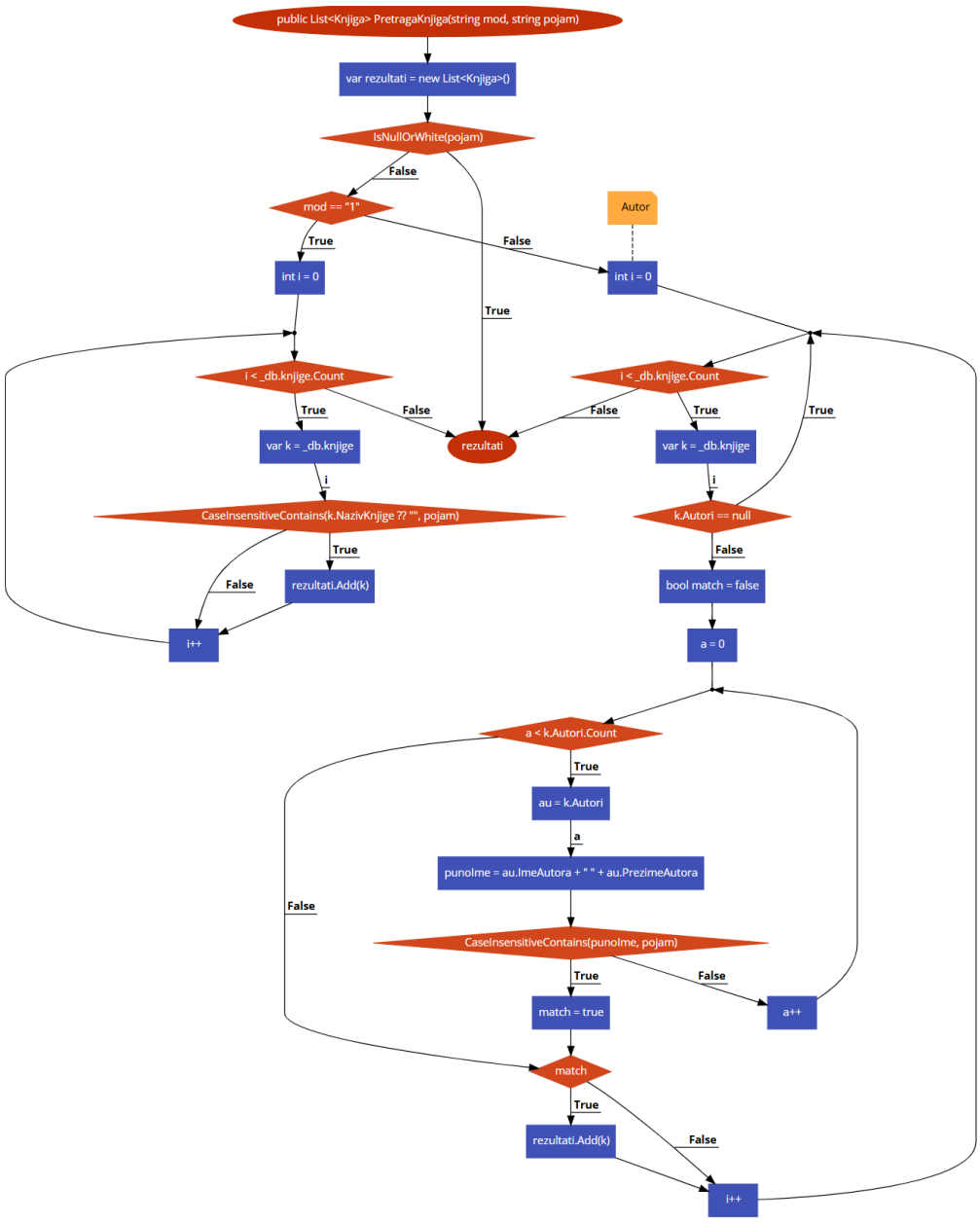
            if (match)
            {
                rezultati.Add(k);
            }
        }
    }
    return rezultati;
}
```

1. Izračunati McCabe metriku i nacrtati kontrolni graf. Uporediti rezultate sa rezultatima iz Code Metrics alata i prodiskutovati kvalitet koda kroz metrike.

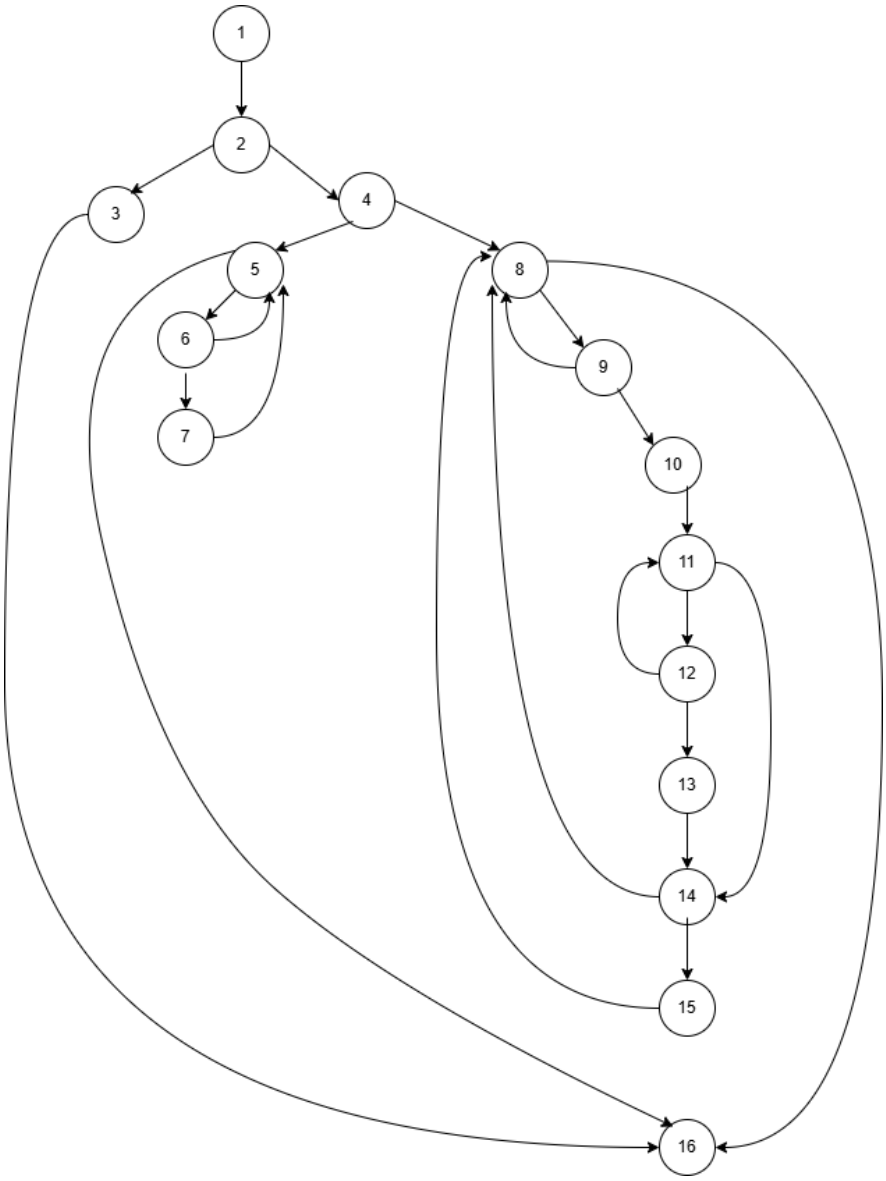
McCabe metrika (ciklomatska kompleksnost)

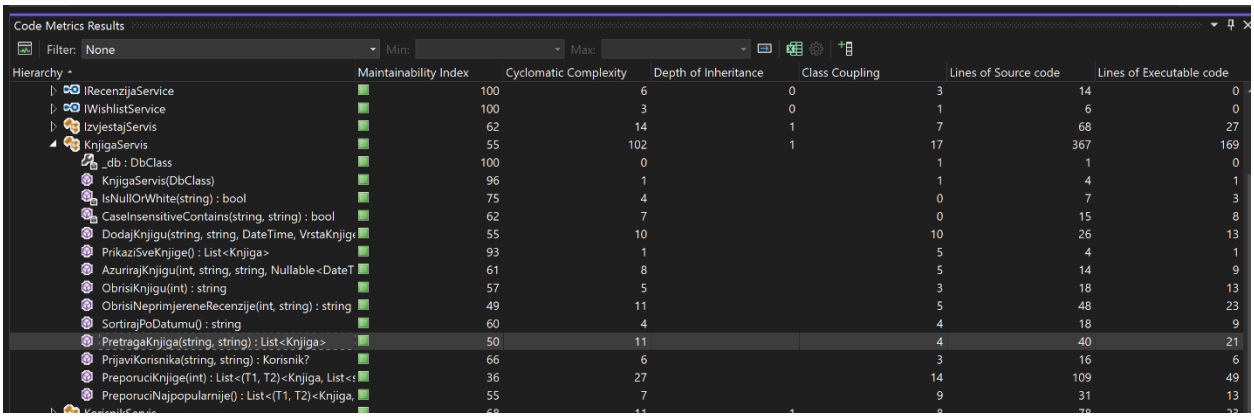
broj grana e = 24
broj čvorova n = 16
broj formiranih neovisnih dijelova grafa p = 1
M = V(G) = e - n + 2p = 24 - 16 + 2 = 10

Kontrolni graf:



Flow graf:





Rezultat se razlikuje za jedinicu u odnosu na naš proračun. Ciklomatska ili McCabeova kompleksnost od 11 ukazuje da metoda ima 11 nezavisnih putanja. Također možemo zaključiti da je indeks održivosti jednak 50, što je poprilično loše. To znači da je metoda još uvijek održiva, ali bi mogla biti puno bolja.

2. Uraditi white box testiranje nad metodom koristeći sve pristupe white box testiranja koja su primjenjiva na metodu. Prikazati odabrane metode, dizajnirane testne slučajeve, i provedeni postupak testiranja.

Tabela puteva na osnovu grafa

Redni broj puta	Čvorovi koje put obuhvata
1	1-2-3-16
2	1-2-4-5-16
3	1-2-4-5-6-7-5-16
4	1-2-4-5-6-5-16
5	1-2-4-8-16
6	1-2-4-8-9-8-16
7	1-2-4-8-9-10-11-14-8-16
8	1-2-4-8-9-10-11-12-13-14-15-8-16
9	1-2-4-8-9-10-11-12-11-12-13-14-15 8-16
10	1-2-4-8-9-10-11-12-11-14-8-16
11	1-2-4-8-9-8-9-10-11-12-13-14-15-8-16

Itd.

Obuhvat iskaza/linija (Line coverage)

Za obuhvat svih linija potrebna su nam 3 puta:

Put 1 (1-2-3-16) - Pojam je prazan, vraća se prazna lista

```
[TestMethod]
0 references
public void PrazanPojam_VracaPraznuListu()
{
    var rezultat = _servis.PretragaKnjiga("1", "");
    Assert.AreEqual(0, rezultat.Count);
}
```

Put 3 (1-2-4-5-6-7-5-16) - Pretraga po nazivu - pronađena knjiga

```
[TestMethod]
0 references
public void PretragaPoNazivu_KnjigaPostoji_VracaKnjigu()
{
    _db.knjige.Add(new Knjiga(1, "123", "Naslov Knjige", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")));
    _db.knjige.Add(new Knjiga(2, "124", "The Hobbit", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")));

    var rezultat = _servis.PretragaKnjiga("1", "naslov");
    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov Knjige"));
}
```

Put 8 (1-2-4-8-9-10-11-12-13-14-15-8-16) - Pretraga po autoru - pronađen autor

```
0 references
public void PretragaPoAutoru_PronadjenAutor()
{
    var autor = CreateAutor("Pero", "Peric", "01.01.1980");
    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")) { Autori = new List<Autor> { autor } };
    var k2 = new Knjiga(2, "124", "Naslov2", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(2, "Izd")) { Autori = new List<Autor> { autor } };
    var k3 = new Knjiga(3, "125", "DrugiAutor", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(3, "Izd")) { Autori = new List<Autor> { CreateAutor("Drugi", "Autor", "01.01.1970") } };
    _db.knjige.Add(k1);
    _db.knjige.Add(k2);
    _db.knjige.Add(k3);

    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(2, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov2"));
}
```

Obuhvat grana/odluka (Branch/Decision Coverage)

Metoda sadrži 9 odluka (if iskazi i petlje). Za postizanje 100% obuhvata grana potrebno je osigurati da se svaka odluka izvrši i sa TRUE i sa FALSE ishodom. Dizajnirano je 6 testnih slučajeva koji pokrivaju sve moguće grane izvršavanja.

PUT 1 (1-2-3-16): Null ili prazan pojam

```
[TestMethod]
0 references
public void PrazanPojam_VracaPraznuListu()
{
    var rezultat = _servis.PretragaKnjiga("1", "");
    Assert.AreEqual(0, rezultat.Count);
}
```

PUT 3 (1-2-4-5-6-7-5-16): Pretraga po nazivu - pronađena knjiga

```
[TestMethod]
0 references
public void PretragaPoNazivu_KnjigaPostoji_VracaKnjigu()
{
    _db.knjige.Add(new Knjiga(1, "123", "Naslov Knjige", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")));
    _db.knjige.Add(new Knjiga(2, "124", "The Hobbit", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")));

    var rezultat = _servis.PretragaKnjiga("1", "naslov");
    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov Knjige"));
}
```

PUT 4 (1-2-4-5-6-5-16): Pretraga po nazivu - nije pronađena knjiga

```
[TestMethod]
0 references
public void PretragaPoNazivu_KnjigaNePostoji_VracaPrazanRezultat()
{
    _db.knjige.Add(new Knjiga(1, "123", "Nalov Knjige", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")));
    _db.knjige.Add(new Knjiga(2, "124", "The Hobbit", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")));

    var rezultat = _servis.PretragaKnjiga("1", "nepostojeca knjiga");

    Assert.AreEqual(0, rezultat.Count);
    Assert.IsFalse(rezultat.Any(k => k.NazivKnjige == "Nalov Knjige"));
    Assert.IsFalse(rezultat.Any(k => k.NazivKnjige == "The Hobbit"));
}
```

PUT 8(1-2-4-8-9-10-11-12-13-14-15-8-16): Pretraga po autoru - pronađen autor (match postaje true)

```
0 references
public void PretragaPoAutoru_PronadjenAutor()
{
    var autor = CreateAutor("Pero", "Peric", "01.01.1980");
    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")) { Autori = new List<Autor> { autor } };
    var k2 = new Knjiga(2, "124", "Naslov2", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(2, "Izd")) { Autori = new List<Autor> { autor } };
    var k3 = new Knjiga(3, "125", "DrugiAutor", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(3, "Izd")) { Autori = new List<Autor> { CreateAutor("Drugi", "Autor", "01.01.1970") } };
    _db.knjige.Add(k1);
    _db.knjige.Add(k2);
    _db.knjige.Add(k3);

    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(2, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov2"));
}
```

PUT 6(1-2-4-8-9-8-16): Pretraga po autoru - autor je null

```
[TestMethod]
0 references
public void PretragaPoAutoru_AutoriNull_PreskaciKnjigu()
{
    var knjiga1 = new Knjiga(1, "123", "Knjiga bez autora", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    { Autori = null
    };
    var autor2 = CreateAutor("J.K.", "Rowling", "31.07.1965");
    var knjiga2 = new Knjiga(2, "124", "Harry Potter", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {Autori = new List<Autor> { autor2 }
    };
    _db.knjige.Add(knjiga1);
    _db.knjige.Add(knjiga2);
    var rezultat = _servis.PretragaKnjiga("2", "rowling");
    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Harry Potter"));
    Assert.IsFalse(rezultat.Any(k => k.NazivKnjige == "Knjiga bez autora"));
}
```

PUT 10(1-2-4-8-9-10-11-12-11-14-8-16): Pretraga po autoru - autor nije pronađen (match ostaje false)

```
[TestMethod]
0 references
public void PretragaPoAutoru_AutorNePostoji_VracaPrazanRezultat()
{
    var autor = CreateAutor("J.K.", "Rowling", "31.07.1965");
    var knjiga = new Knjiga(1, "123", "Harry Potter", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {
        Autori = new List<Autor> { autor }
    };
    _db.knjige.Add(knjiga);
    var rezultat = _servis.PretragaKnjiga("2", "nepostojeći");
    Assert.AreEqual(0, rezultat.Count);
    Assert.IsFalse(rezultat.Any(k => k.NazivKnjige == "Harry Potter"));
}
```

Obuhvat uslova (Condition Coverage)

Pošto svi if iskazi u metodi sadrže jednostavne uslove, obuhvat uslova je identičan obuhvatu grana. Svaki uslov se testira u TRUE i FALSE stanju kroz isti skup od 6 testnih slučajeva.

Obuhvat puteva (Path coverage)

Kao što je u predavanju naglašeno, ovaj koncept nije praktičan u mnogim slučajevima jer je potrebno mnogo resursa i testnih slučajeva za njegovo izvršavanje**.** U odabranoj metodi postoje tri petlje i postoji mnogo različitih puteva koje bismo trebali testirati.

Putevi koji nisu obuhvaćeni prethodnim testovima

PUT 2(1-2-4-5-16): Pretraga po nazivu - prazna baza

PUT 7(1-2-4-8-9-10-11-14-8-16): Pretraga po autoru - prazna lista autora

Obuhvat petlji (Loop coverage)

Metoda sadrži tri petlje:

- spoljašnju petlju za pretragu po nazivu knjige,
- spoljašnju petlju za pretragu po autoru,
- unutrašnju petlju za iteriranje kroz autore knjige.

Formirat ćemo za prvu petlju sljedeće testne slučajeve:

- Preskočiti unutrašnjost petlje (_db.knjige.Count = 0)
- Jedan prolazak kroz petlju (_db.knjige.Count = 1)
- Dva prolaska kroz petlju (_db.knjige.Count = 2)
- m prolazaka kroz petlju (_db.knjige.Count = 5, m=3)
- n - 1 prolazaka kroz petlju (_db.knjige.Count = 5)
- n prolazaka kroz petlju (_db.knjige.Count = 5)

```
[TestMethod]
0 references
public void Test_Petlja_PreskociUnutrasnjost()
{
    _db.knjige.Clear();

    var rezultat = _servis.PretragaKnjiga("1", "naslov");

    Assert.AreEqual(0, rezultat.Count);
}

[TestMethod]
0 references
public void Test_Petlja_JedanProlaz()
{
    _db.knjige.Clear();
    _db.knjige.Add(new Knjiga(1, "123", "MojNaslov", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")));

    var rezultat = _servis.PretragaKnjiga("1", "naslov");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "MojNaslov"));
}
```

```
[TestMethod]
0 references
public void Test_Petlja1_DvaProlaza()
{
    _db.knjige.Clear();
    _db.knjige.Add(new Knjiga(1, "123", "MojNaslov", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd")));
    _db.knjige.Add(new Knjiga(2, "124", "DrugiNaslov", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(2, "Izd")));

    var rezultat = _servis.PretragaKnjiga("1", "DrugiNaslov");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "DrugiNaslov"));
}
```

```
[TestMethod]
0 references
public void Test_Petlja1_MManjeOdN_MatchNa3()
{
    _db.knjige.Clear();
    //n=5, m=3
    for (int i = 1; i <= 5; i++)
    {
        _db.knjige.Add(new Knjiga(i, $"12{i}", $"Naslov{i}", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(i, "Izd")));
    }

    var rezultat = _servis.PretragaKnjiga("1", "naslov3");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov3"));

}
```

```
public void Test_Petlja1_NMinus1Prolaza()
{
    _db.knjige.Clear();
    for (int i = 1; i <= 5; i++)
    {
        _db.knjige.Add(new Knjiga(i, $"12{i}", $"Naslov{i}", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(i, "Izd")));
    }

    var rezultat = _servis.PretragaKnjiga("1", "naslov4");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov4"));
}
```

```
public void Test_Petlja1_NProlaza()
{
    _db.knjige.Clear();

    for (int i = 1; i <= 5; i++)
    {
        _db.knjige.Add(new Knjiga(i, $"12{i}", $"Naslov{i}", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(i, "Izd")));
    }
    var rezultat = _servis.PretragaKnjiga("1", "naslov5");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov5"));
}
```

Druga for petlja je ugnježdene petlje u grani pretraga po autoru.

Formiramo testne slučajeve:

- Preskočiti unutrašnjost petlje
  _db.knjige.Count = 1
  k.Autori.Count = 0
- Jedan prolazak kroz petlju
  _db.knjige.Count = 1
  k.Autori.Count = 1
- Dva prolaska kroz petlju
  _db.knjige.Count = 1
  k.Autori.Count = 2
- m prolazaka kroz petlju
  _db.knjige.Count = 1
  k.Autori.Count = 5, m=3
- n-1 prolaza kroz petlju i n prolaza kroz petlju

  _db.knjige.Count = 1
  k.Autori.Count = 5

```
[TestMethod]
0 references
public void Test_Unutrasnja_PreskociUnutrasnjost_AutoriPrazni()
{
    _db.knjige.Clear();

    // vanjska petlja: 1 prolaz (1 knjiga)
    // unutrasnja petlja: 0 prolaza (Autori = [])
    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {
        Autori = new List<Autor>()
    };
    _db.knjige.Add(k1);

    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(0, rezultat.Count);
}

[TestMethod]
0 references
public void Test_Unutrasnja_JedanProlaz_AutorCount1()
{
    _db.knjige.Clear();
    var autor = CreateAutor("Pero", "Peric", "01.01.1980");
    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {
        Autori = new List<Autor> { autor }
    };
    _db.knjige.Add(k1);

    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
}
```

```
[TestMethod]
0 references
public void Test_Unutrasnja_DvaProlaza_AutorCount2()
{
    _db.knjige.Clear();
    var a1 = CreateAutor("Mujo", "Mujic", "01.01.1970");
    var a2 = CreateAutor("Pero", "Peric", "01.01.1980");

    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {
        Autori = new List<Autor> { a1, a2 }
    };
    _db.knjige.Add(k1);

    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
}
```

```
[TestMethod]
0 references
public void Test_Unutrasnja_MManjeOdN_MatchNa3()
{
    _db.knjige.Clear();
    var autori = new List<Autor>
        {
            CreateAutor("A1", "B1", "01.01.1970"),
            CreateAutor("A2", "B2", "01.01.1970"),
            CreateAutor("Pero", "Peric", "01.01.1980"),
            CreateAutor("A3", "B3", "01.01.1970"),
            CreateAutor("A4", "B4", "01.01.1970")
        };

    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {
        Autori = autori
    };
    _db.knjige.Add(k1);

    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
}
```

```
[TestMethod]
0 references
public void Test_Unutrasnja_NMinus1Prolaza()
{
    _db.knjige.Clear();
    var autori = new List<Autor>
        {
            CreateAutor("A1", "B1", "01.01.1970"),
            CreateAutor("A2", "B2", "01.01.1970"),
            CreateAutor("A3", "B3", "01.01.1970"),
            CreateAutor("Pero", "Peric", "01.01.1980"),
            CreateAutor("A4", "B4", "01.01.1970")
        };

    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {
        Autori = autori
    };
    _db.knjige.Add(k1);

    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
}
```

```
[TestMethod]
0 references
public void Test_Unutrasnja_NProlaza()
{
    _db.knjige.Clear();
    var autori = new List<Autor>
        {
            CreateAutor("A1", "B1", "01.01.1970"),
            CreateAutor("A2", "B2", "01.01.1970"),
            CreateAutor("A3", "B3", "01.01.1970"),
            CreateAutor("A4", "B4", "01.01.1970"),
            CreateAutor("Pero", "Peric", "01.01.1980")
        };

    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {
        Autori = autori
    };
    _db.knjige.Add(k1);
    var rezultat = _servis.PretragaKnjiga("2", "pero peric");
    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
}
```

Isto ćemo uraditi i za vanjsku petlju.

```
[TestMethod]
0 references
public void Test_VanjskaPetlja_PreskociUnutrasnjost()
{
    _db.knjige.Clear();
    var rezultat = _servis.PretragaKnjiga("2", "pero peric");
    Assert.AreEqual(0, rezultat.Count);
}

[TestMethod]
0 references
public void Test_VanjskaPetlja_JedanProlaz()
{
    _db.knjige.Clear();

    // vanjska petlja: 1 prolaz (1 knjiga)
    // unutrasnja: minimalno (1 autor)
    var autor = CreateAutor("Pero", "Peric", "01.01.1980");
    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {
        Autori = new List<Autor> { autor }
    };
    _db.knjige.Add(k1);

    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(1, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
}
```

```
[TestMethod]
| 0 references
public void Test_VanjskaPetlja_DvaProlaza()
{
    _db.knjige.Clear();

    var autor = CreateAutor("Pero", "Peric", "01.01.1980");

    var k1 = new Knjiga(1, "123", "Naslov1", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(1, "Izd"))
    {
        Autori = new List<Autor> { autor }
    };
    var k2 = new Knjiga(2, "124", "Naslov2", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(2, "Izd"))
    {
        Autori = new List<Autor> { autor }
    };

    _db.knjige.Add(k1);
    _db.knjige.Add(k2);

    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(2, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov2"));
}
```

```
[TestMethod]
| 0 references
public void Test_VanjskaPetlja_MProlaza()
{
    _db.knjige.Clear();

    var autor = CreateAutor("Pero", "Peric", "01.01.1980");

    for (int i = 1; i <= 5; i++)
    {
        _db.knjige.Add(new Knjiga(i, $"12{i}", $"Naslov{i}", DateTime.Now, VrstaKnjige.NOVEL, new Izdavac(i, "Izd"))
        {
            Autori = new List<Autor> { autor }
        });
    }
    var rezultat = _servis.PretragaKnjiga("2", "pero peric");

    Assert.AreEqual(5, rezultat.Count);
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov1"));
    Assert.IsTrue(rezultat.Any(k => k.NazivKnjige == "Naslov5"));
}
```

Obuhvat toka podataka (Data flow coverage)

PUT 1(1-2-3-16) - Pojam je prazan, vraća se prazna lista (var rezultati = new List<Knjiga>();)

PUT 3(1-2-4-5-6-7-5-16) - Pretraga po nazivu - pronađena knjiga(k.NazivKnjige, rezultati.Add(k))

PUT 4(1-2-4-5-6-5-16) - Pretraga po nazivu - nije pronađena knjiga(k.NazivKnjige)

PUT 10(1-2-4-8-9-10-11-12-11-14-8-16)-Pretraga po autoru - autor nije pronađen (match ostaje false)

PUT 8(1-2-4-8-9-10-11-12-13-14-15-8-16): Pretraga po autoru - pronađen autor (match postaje true)

3. Izvršiti code tuning za odabranu metodu primjenom minimalno 3 različite tehnike. Navesti koje su tehnike korištene i prikazati novi kod. Nakon svake urađene tehnike zabilježiti vrijeme izvršavanja i alociranu memoriju sa velikim brojem pokretanja. Zabilježiti i prodiskutovati i McCabe metriku i indeks održavanja te metode prije i poslije code tuning-a.

```
public List<Knjiga> PretragaKnjiga(string mod, string pojam)
{
    var rezultati = new List<Knjiga>();
    if (IsNullOrWhiteSpace(pojam)) { return rezultati; }
    if (mod == "1")
    {
        for (int i = 0; i < _db.knjige.Count; i++)
        {
            var k = _db.knjige[i];
            if (CaseInsensitiveContains(k.NazivKnjige ?? "", pojam))

            {
                rezultati.Add(k);
            }
        }
    }
    else // Autor

    {
        for (int i = 0; i < _db.knjige.Count; i++)
        {
            var k = _db.knjige[i];
            if (k.Autori == null) continue;
            bool match = false;

            for (int a = 0; a < k.Autori.Count; a++)
            {
                var au = k.Autori[a];
                string punoIme = $"{au.ImeAutora} {au.PrezimeAutora}";
                if (CaseInsensitiveContains(punoIme, pojam))

                { match = true; break; }
            }
            if (match)
            {
                rezultati.Add(k);
            }
        }
    }
    return rezultati;
}
```

Prije code tuning-a, funkcija je pokrenuta veliki broj puta (10000 puta) da bi se mogli uporediti rezultati performansi prije code tuning-a i poslije:

```
Vrijeme (10.000 pokretanja): 3220 ms
Alocirana memorija: 10013.43 KB
```

1. Minimiziranje rada unutar petlje

U okviru prve tehnike code tuning-a primijenjeno je minimiziranje rada unutar petlji, pri čemu su često korištene vrijednosti (lista knjiga, broj elemenata kolekcija i reference na objekte) keširane u lokalne varijable, kako bi se smanjio broj ponovljenih dereferenciranja i poziva unutar petlji.

```
public List<Knjiga> PretragaKnjiga(string mod, string pojam)
{
    var rezultati = new List<Knjiga>();
    if (IsNullOrWhite(pojam)) return rezultati;

    var knjige = _db.knjige;
    int n = knjige.Count;

    if (mod == "1")
    {
        for (int i = 0; i < n; i++)
        {
            var k = knjige[i];
            var naziv = k.NazivKnjige ?? "";
            if (CaseInsensitiveContains(naziv, pojam))
                rezultati.Add(k);
        }
    }
    else
    {
        for (int i = 0; i < n; i++)
        {
            var k = knjige[i];
            var autori = k.Autori;
            if (autori == null) continue;

            bool match = false;
            int m = autori.Count;

            for (int a = 0; a < m; a++)
            {
                var au = autori[a];
                string punoIme = $"{au.ImeAutora} {au.PrezimeAutora}";
                if (CaseInsensitiveContains(punoIme, pojam))
                { match = true; break; }
            }

            if (match) rezultati.Add(k);
        }
    }
    return rezultati;
}
```

U poređenju sa početnom (baseline) verzijom, uočen je pad vremena izvršavanja, dok je potrošnja memorije ostala na sličnom nivou

```
Vrijeme (10.000 pokretanja): 2828 ms
Alocirana memorija: 10013.43 KB
```

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
PrikaziSveKnjige() : List<Knjiga>	93	1		5	4	1
ObrisiKnjigu(int, string, string, Nullable<DateTi	61	8		5	14	9
ObrisiKnjigu(int) : string	57	5		3	18	13
ObrisiNeprimjereneRecenzije(int, string) : string	49	11		5	48	23
SortirajPoDatumu() : string	60	4		4	18	9
PretragaKnjiga(string, string) : List<Knjiga>	49	11		4	42	23
PrijaviKorisnika(string, string) : Korisnik?	66	6		3	16	6
PreporuciKnjige(int) : List<(T1, T2)<Knjiga, List<st	36	27		14	109	49
PreporuciNajpopularnije() : List<(T1, T2)<Knjiga, i	55	7		9	31	13
IzvjestajServis	62	14	1	7	68	27
IWishlistService	100	3	0	1	6	0

Analizom Code Metrics alata možemo zaključiti da se indeks održavanja smanjio za jedan dok je ciklomatska kompleksnost ostala isti tako da se i nije neka velika promjena desila ovom tehnikom.

2. Tuning stringova (bez ToLower alokacija)

```
private static bool ContainsIgnoreCase(string text, string pattern)
{
    if (text == null) return false;
    return text.IndexOf(pattern, StringComparison.OrdinalIgnoreCase) >= 0;
}
30 references | 28/28 passing
public List<Knjiga> PretragaKnjiga(string mod, string pojam)
{
    var rezultati = new List<Knjiga>();
    if (string.IsNullOrWhiteSpace(pojam)) return rezultati;

    var knjige = _db.knjige;
    int n = knjige.Count;

    if (mod == "1")
    {
        for (int i = 0; i < n; i++)
        {
            var k = knjige[i];
            var naziv = k.NazivKnjige ?? "";
            if (ContainsIgnoreCase(naziv, pojam))
                rezultati.Add(k);
        }
    }
    else
    {
        for (int i = 0; i < n; i++)
        {
            var k = knjige[i];
            var autori = k.Autori;
            if (autori == null) continue;

            bool match = false;
            for (int a = 0; a < autori.Count; a++)
            {
                var au = autori[a];
                string punoIme = $"{au.ImeAutora} {au.PrezimeAutora}";
                if (ContainsIgnoreCase(punoIme, pojam))
                { match = true; break; }
            }
            if (match) rezultati.Add(k);
        }
    }

    return rezultati;


}
```

U okviru druge tehnike koristimo tuning stringova tjs. izbacujemo CaseInsensitiveContains (ToLower() stvara nove stringove i povećava alokacije) koristimo IndexOf(..., StringComparison.OrdinalIgnoreCase) (bez novih stringova)

Code Metrics Results

Filter: None Min: Max:

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Source code	Lines of Executable code
SortirajPoDatumu() : string	60	4		4	18	9
ContainsIgnoreCaseNova(string, string) : bool	82	2		1	5	2
PretragaKnjiga(string, string) : List<Knjiga>	50	11		4	43	22
PrijaviKorisnika(string, string) : Korisnik?	66	6		3	16	6
PreporuciKnjige(int) : List<(T1, T2)<Knjiga, List	36	27		14	109	49
PreporuciNajpopularnije() : List<(T1, T2)<Knjig	55	7		9	31	13
IzvjestajServis	62	14	1	7	68	27

```
Vrijeme (10.000 pokretanja): 504 ms
Alocirana memorija: 9248.70 KB
```

U poređenju sa prethodnom verzijom, vidimo smanjenje alocirane memorije, dok se vrijeme izvršavanja značajno smanjilo. Analiza Code Metrics alata pokazuje da se vrijednost Maintainability Index povećala za 1.

3. Eliminacija nepotrebnih izraza

U okviru druge tehnike code tuning-a izvršena je eliminacija nepotrebnih izraza unutar unutrašnje petlje, pri čemu je izbjegnuto formiranje privremenog stringa koji predstavlja puno ime autora. Umjesto toga, izvršena je direktna provjera imena i prezimena autora. Na ovaj način smanjene su nepotrebne alokacije memorije i poboljšane performanse metode.

```
public List<Knjiga> PretragaKnjiga(string mod, string pojam)
{
    var rezultati = new List<Knjiga>();
    if (string.IsNullOrWhiteSpace(pojam)) return rezultati;

    var knjige = _db.knjige;
    int n = knjige.Count;

    if (mod == "1")
    {
        for (int i = 0; i < n; i++)
        {
            var k = knjige[i];
            var naziv = k.NazivKnjige ?? "";
            if (ContainsIgnoreCase(naziv, pojam))
                rezultati.Add(k);
        }
    }
    else
    {
        for (int i = 0; i < n; i++)
        {
            var k = knjige[i];
            var autori = k.Autori;
            if (autori == null || autori.Count == 0) continue;

            bool match = false;

            for (int a = 0; a < autori.Count; a++)
            {
                var au = autori[a];

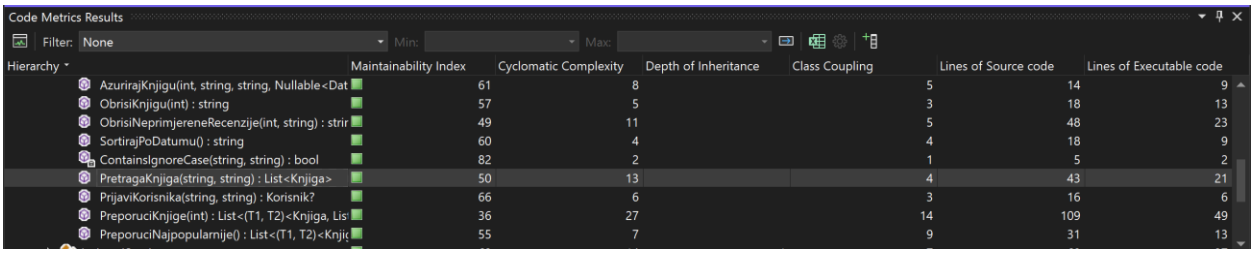
                if (ContainsIgnoreCase(au.ImeAutora, pojam) || ContainsIgnoreCase(au.PrezimeAutora, pojam))
                { match = true; break; }
            }

            if (match) rezultati.Add(k);
        }
    }

    return rezultati;
}
```

```
Vrijeme (10.000 pokretanja): 429 ms
Alocirana memorija: 337.57 KB
```

U poređenju sa prethodnom verzijom, vidimo značajno smanjenje alocirane memorije, što potvrđuje da je eliminacija nepotrebnih string alokacija imala direktan pozitivan efekat na memorijsku efikasnost metode.



Analiza Code Metrics alata pokazuje da je cyclomatic complexity blago povećana (sa 11 na 13), što je posljedica dodatnog logičkog uslova (OR) unutar petlje. Ipak, Maintainability Index ostaje kao na početku(50).

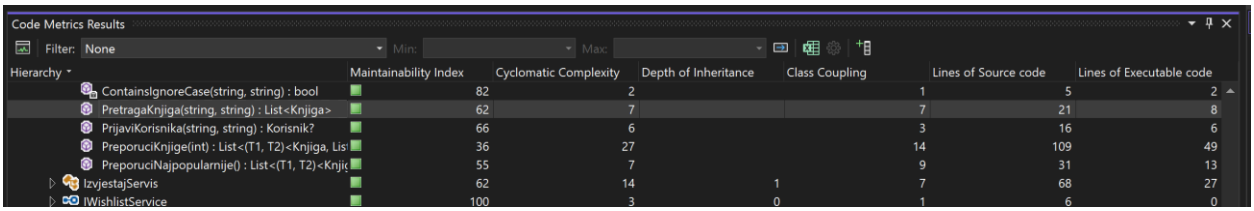
4. LINQ (Language Integrated Query) je tehnologija u C#-u i .NET-u koja omogućava pisanje upita za kolekcije podataka direktno u kodu na deklarativan i intuitivan način. Iskoristit ćemo I ovu tehniku, jer nismo uspjeli postići bolje metrike prethodnim tehnikama.

```
public List<Knjiga> PretragaKnjiga(string mod, string pojam)
{
    if (string.IsNullOrWhiteSpace(pojam))
        return new List<Knjiga>();

    var knjige = _db.knjige;

    if (mod == "1")
    {
        return knjige
            .Where(k => ContainsIgnoreCase(k.NazivKnjige ?? "", pojam))
            .ToList();
    }

    return knjige
        .Where(k => k.Autori != null && k.Autori.Count > 0 &&
                    k.Autori.Any(au =>
                        ContainsIgnoreCase(au.ImeAutora, pojam) ||
                        ContainsIgnoreCase(au.PrezimeAutora, pojam)))
        .ToList();
}
```



```
Vrijeme (10.000 pokretanja): 727 ms
Alocirana memorija: 1951.35 KB
```

Analizom Code Metrics alata uočeno je smanjenje ciklomatske kompleksnosti metode sa 13 na 7, dok je Maintainability Index porastao na vrijednost 62, što ukazuje na poboljšanu održivost i čitljivost koda. Smanjen je i broj izvršivih linija koda, čime je dodatno pojednostavljena struktura metode.

4. Nakon code tuning-a, izvršiti refaktoring tako da se postignu što bolje vrijednosti metrika. Prikazati kojoj stavki unutar checklist-e za refaktoring svaka promjena pripada, i kako ona utječe na metrike u sklopu Code Metrics.

U refaktorisanju metode koristila sam sljedeće stavke iz checkliste sa predavanja:

Checklist stavka: *Routine Level Refactorings - Separate routines whose behavior depends on parameters passed in*

Checklist stavka: *Statement Level Refactorings - Move a complex boolean expression into a well-named boolean function*

Checklist stavka: *Data Level Refactorings - Rename a variable with a clearer or more informative name*

Nakon refaktoringa metoda izgleda ovako:

```
public List<Knjiga> PretragaKnjiga(string mod, string pojam)
{
    if (string.IsNullOrWhiteSpace(pojam))
        return new List<Knjiga>();

    return mod == "1"
        ? PretragaPoNazivu(pojam)
        : PretragaPoAutoru(pojam);
}
```

```
private static bool ContainsIgnoreCase(string text, string pattern)
{
    if (text == null) return false;
    return text.IndexOf(pattern, StringComparison.OrdinalIgnoreCase) >= 0;
}

1 reference
private static bool AutoriSadrzePojam(List<Autor>? autori, string pojam)
{
    if (autori == null) return false;

    return autori.Any(autor => ContainsIgnoreCase(autor.ImeAutora, pojam) || ContainsIgnoreCase(autor.PrezimeAutora, pojam));
}

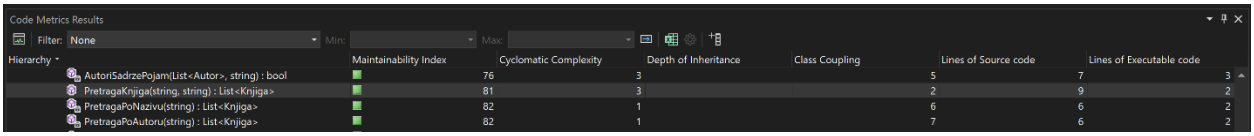
39 references | 35/35 passing
public List<Knjiga> PretragaKnjiga(string mod, string pojam)
{
    if (string.IsNullOrWhiteSpace(pojam))
        return new List<Knjiga>();

    return mod == "1"
        ? PretragaPoNazivu(pojam)
        : PretragaPoAutoru(pojam);
}

1 reference
private List<Knjiga> PretragaPoNazivu(string pojam)
{
    return _db.knjige
        .Where(knjiga => ContainsIgnoreCase(knjiga.NazivKnjige, pojam))
        .ToList();
}

1 reference
private List<Knjiga> PretragaPoAutoru(string pojam)
{
    return _db.knjige
        .Where(knjiga => AutoriSadrzePojam(knjiga.Autori, pojam))
        .ToList();
}
```

CodeMetrics nakon refaktoringa izgleda ovako:



Možemo vidjeti, nakon refaktoringa, indeks održavanja povećao na 81, a ciklomatska kompleksnost dodatno smanjila na 3. Tako zaključujemo da smo dobili najbolje moguće rezultate metrika, što je i bio krajnji cilj postići.